



Universidad Nacional de Colombia

Lenguajes de programación

Tarea 17: Aritmética de punto flotante para la máquina construida en la Tarea 10.

Jorge Eduardo Ortiz Triviño

Stephan Alain Roland García Martiquet
Omar Jhoan Ospina Sanchez
Sebastian Contreras Tellez
Camilo Trujillo Garzon
Daniel Santiago Rodriguez Rodriguez
Juliana Parra Caro
Cristian Camilo Ochica Maldonado

Fecha: Lunes 5 de octubre de 2026

1. Marco teórico

En el diseño de computadores es importante la unidad central de procesamiento denominada CPU por sus iniciales, pues requiere mecanismos especializados que manejen datos reales o fraccionarios. La ALU por su parte opera principalmente con números enteros en complemento a dos, el procesamiento eficiente de números decimales de alta precisión la hace la unidad de punto flotante o FPU, para esto se requiere la integración de instrucciones que permitan a la máquina realizar operaciones como la suma resta, multiplicación y división de flotantes.

Para la representación de números reales de manera estandarizada en sistemas digitales es importante que un emulador implemente el estándar IEEE 754 de doble precisión, este formato ocupa 64 bits que se estructuran de la siguiente forma:

- Bit de Signo: El primer bit de izquierda a derecha determina si el número es positivo (0) o negativo (1).
- Exponente: Usa 11 bits para almacenar la magnitud del exponente desplazado con un sesgo de 1023 bits, esto permite representar exponentes positivos y negativos sin tener que hacer uso de un bit de signo independiente para el exponente.
- Mantisa: Los 52 bits restantes representan los dígitos significativos del número.

Ejemplos

El valor hexadecimal 0x3FF8000000000000 corresponde al número 1.5.

El valor hexadecimal 0x4020000000000000 corresponde al número 8.0.

Para realizar operaciones aritméticas como la suma o la resta de punto flotante se debe seguir el siguiente procedimiento, pues al tratarse de un formato estándar específico no se puede tomar como una operación lineal simple

1. **Alineación de exponentes:** Se comparan los exponentes de ambos operandos, el número con el exponente menor desplaza su mantisa hacia la derecha hasta que ambos exponentes coincidan.
2. **Suma de mantisas:** Cuando se encuentren alineados se pueden sumar o restar teniendo en cuenta el bit implícito de las mantisas.
3. **Normalización:** El resultado de la operación se ajusta para que mantenga el formato estándar, colocando un 1 a la izquierda del punto binario.

Aplicar el estándar IEEE 754 a una máquina permite realizar cálculos complejos con números de punto flotante, por ejemplo se puede realizar el cálculo la raíz cuadrada de un número real x usando técnicas algorítmicas iterativas, el método de Herón o método babilónico supone una forma de hallar raíces cuadradas que parte de una estimación inicial de y_0 para el valor de la raíz de x . A medida que el valor de 0 en y aumente por ejemplo hasta un paso n , la mejor estimación (como la $n+1$) se puede ir obteniendo promediando y_n y el cociente de x/y_n .

$$y_{n+1} = \frac{1}{2} \left(y_n + \frac{x}{y_n} \right)$$

Otra aplicación computacional utilizando el estándar IEEE 754 es encontrar la constante de Brun, la cual es la suma de los inversos de todos los primos gemelos (parejas de primos cuya diferencia es exactamente 2) y se usa para estudiar la distribución y densidad de estos números. Mientras que la suma de los inversos de todos los primos diverge creciendo hasta el infinito la serie de los primos gemelos converge hasta un número finito que adquiere el valor aproximado de 1,90216 demostrado que los primos gemelos se encuentran en menor cantidad que los primos en general.

$$B_2 = \sum \left(\frac{1}{p} + \frac{1}{p+2} \right)$$

2. Justificación

Teniendo en cuenta que el emulador original desarrollado en la Tarea 10 no consta con una unidad aritmética que le permita realizar operaciones en punto flotante y además su repertorio de instrucciones se encuentra de cierta manera limitado posibilitando al usuario operar únicamente con números enteros. La implementación de una unidad de punto flotante o FPU que cumpla el estándar IEEE 754 es importante para ampliar las capacidades del sistema y permitir que la arquitectura original de 64 bits sea aprovechada en su totalidad y permita el procesamiento de problemas matemáticos de un espectro más amplio que requieren alta precisión o un manejo de rangos específicos.

Además realizar la programación de rutinas o procesos como el desempaquetamiento de los bits, la alineación de los exponentes y el manejo de excepciones para ceros o valores infinitos supone un aprendizaje avanzado sobre el funcionamiento a bajo nivel de las máquinas y todo el trabajo que hay detrás de una operación que puede resultar tan básica como sumar $1.5 + 0.8$. Todo esto aplicado a algoritmos iterativos como el cálculo de raíces mediante el método de Herón o encontrar la constante de Brun permite desarrollar una mayor robustez en el emulador.

3. Diseño de la aplicación

Los 16 registros usados en el emulador se mantienen y la siguiente es la adaptación a las rutinas del estándar IEEE 754

- **R0:** Registro principal de entrada y salida.
- **R1:** Almacena el bit de signo.
- **R2:** Almacena el exponente codificado.

- **R3:** Contiene el exponente real sin sesgo.
- **R4:** Almacena la fracción de 52 bits.
- **R5:** Contiene el significando de 53 bits con bit implícito normalizado.
- **R6:** Registro de clasificación estructural para validar si el número es cero, normal, infinito o NaN.
- **R7 a R12:** Registros temporales y auxiliares para operaciones internas.
- **R13:** Puntero de pila (*Stack Pointer* - SP), operando en el rango superior de la memoria RAM.
- **R14 y R15:** Reservados para temporales de bajo nivel y enlaces de retorno.

Como el cargador inicial diseñado para el emulador procesa únicamente palabras hexadecimales usando los formatos de 64 bits establecidos previamente se diseñó en el lenguaje Python un ensamblador que reconozca el mnemónico y lo pueda emular. Este ensamblador elimina los comentarios y líneas que inician con # y se encuentran únicamente con el fin de guiar al usuario pero no aportan a la funcionalidad del programa, luego detecta etiquetas de salto y calcula las direcciones usando el program counter (PC). Luego usa las instrucciones .quad para inyectar constantes numéricas de 64 bits en la memoria siempre y cuando se encuentre en formato hexadecimal. Finalmente hace la traducción de las instrucciones válidas por los formatos anteriormente definidos en el ISA.

Tabla de instrucciones agregadas para la operación de flotantes de doble precisión.

Opcode	Instrucción	Descripción	Registros Necesarios / Formato
0x0061	FADD	Suma dos números en formato de doble precisión.	Formato Tipo-R: rd, ra, rb
0x0062	FSUB	Resta dos números en formato de doble precisión.	Formato Tipo-R: rd, ra, rb
0x0063	FMUL	Multiplica dos números en formato de doble precisión.	Formato Tipo-R: rd, ra, rb

0x0064	FDIV	Divide dos números en formato de doble precisión.	Formato Tipo-R: rd, ra, rb
0x0065	FSQRT	Calcula la raíz cuadrada de un número en doble precisión.	Formato Tipo-R: rd, ra
0x0066	FUNPACK	Descompone los componentes internos de un número flotante (signo, exponente y fracción) distribuyéndolos en registros auxiliares (R1 a R6).	Formato Tipo-R: ra
0x0067	FHERON	Ejecuta un ciclo de aproximación de la raíz cuadrada por el método de Herón utilizando un registro contador dinámico.	Formato Tipo-R: rd, ra, rc
0x0068	FITOF	Convierte un número entero sin signo a formato flotante de doble precisión..	Formato Tipo-R: rd, ra

4. Código fuente

El código fuente se encuentra en el archivo comprimido 17 Ospina Sanchez Omar Jhoan 02.zip en la misma carpeta en la que se ubica este documento.

5. Manual de usuario

Teniendo en cuenta que en la tarea 10 se presentó el manual de usuario del emulador para evitar la repetición innecesaria de información solo se va a presentar lo relacionado directamente con el módulo de ensamblador y la extensión hecha para el acoplamiento del estándar IEEE 754.

Requisitos

- Intérprete de Python versión 3.8 o superior instalada.
- Los archivos del emulador completos dentro de la carpeta del proyecto sumado a los archivos .asm que contienen los ejercicios de prueba de funcionamiento del módulo ensamblador.

Con los requisitos cumplidos, se debe ejecutar el archivo main del proyecto y se verá en consola el menú principal del emulador, al seleccionar la opción 1 se permitirá cargar un archivo de código previamente definido, va a ser necesario escribir el nombre exacto del archivo y su extensión (ejemplo: brun.asm). Si el archivo se carga correctamente se podrá ver la ejecución paso a paso o el resultado final de los registros, banderas y el estado de la CPU (se recomienda seleccionar la ejecución automática (opción 5) para evitar avanzar manualmente la gran cantidad de iteraciones que requieren ciertos códigos como el de la constante de brun). Cuando en consola se vea que la bandera *Halted* cambió su estado a *True* la ejecución del programa será completada.

6. Manual técnico

El emulador usa un banco de 16 registros de 64 bits cada uno, como se observa en el diseño de la aplicación se enlistan los registros adaptados al estándar IEEE 754 para su correcto funcionamiento e incorpore la operabilidad de doble precisión de manera adecuada usando las instrucciones definidas previamente. El ensamblador se realiza al igual que todo el proyecto en lenguaje Python para fácil adaptación, por su parte el estándar 754 se realiza directamente en assembler para constatar los registros usados y su integridad real con el emulador, a continuación se presenta una lista de la manera en que se almacenan las constantes IEEE 754 en memoria cuyas direcciones comprenden desde la 0x100 hasta la 0x130.

- **0x100 (0x8000000000000000):** Máscara de aislamiento del bit de signo.
- **0x108 (0x7FF0000000000000):** Máscara del exponente almacenado.
- **0x110 (0x000FFFFFFFFFFFFFFF):** Máscara de la fracción de 52 bits.
- **0x118 (0x0010000000000000):** Valor del bit implícito normalizado (2^{52}).
- **0x120 (0x00000000000003FF):** Constante del sesgo de exponente (1023).
- **0x128 (0x3FE0000000000000):** Constante flotante equivalente a 0.5.
- **0x130 (0xFFFFFFFFFFFFC02):** Valor entero de -1022.

7. Análisis de resultados

Estimación de la constante de Brun

```
LOAD R2, ZERO
LDI R12, 1000
LDI R14, 3
LDI R15, 0
```

BRUN_LOOP:

```
CMP R11, R14, R12
JG BRUN_DONE
MOV R0, R14
CALL IS_PRIME
CMP R11, R1, R15
JZ BRUN_NEXT
```

```
MOV R0, R14
ADDI R0, 2
CALL IS_PRIME
CMP R11, R1, R15
JZ BRUN_NEXT
```

```
FITOF R5, R14
MOV R0, R14
ADDI R0, 2
FITOF R6, R0
LOAD R9, ONE
FDIV R7, R9, R5
FDIV R8, R9, R6
FADD R2, R2, R7
FADD R2, R2, R8
```

BRUN_NEXT:

```
MOV R0, R14
```

```
ADDI R0, 2
MOV R14, R0
JMP BRUN_LOOP
```

```
BRUN_DONE:
    OUT R2, 2
    HLT
```

```
IS_PRIME:
    LDI R1, 1
    LDI R3, 2
    LDI R7, 0
```

```
PRIME_LOOP:
    MUL R4, R3, R3
    CMP R6, R4, R0
    JG PRIME_RETURN
    MOD R5, R0, R3
    CMP R6, R5, R7
    JZ PRIME_COMPOSITE
    INC R3
    JMP PRIME_LOOP
```

```
PRIME_COMPOSITE:
    LDI R1, 0
PRIME_RETURN:
    RET
```

```
ZERO:
    .quad 0x0000000000000000
ONE:
    .quad 0x3FF0000000000000
```

```

=====
ESTADO DE LA CPU
=====
PC: 0x2F0 | IR: 0x1200000000000000 | Halted: True
MAR: 0x2E8 | MDR: 0x1200000000000000
Banderas (FR) -> Z: 0 | N: 0 | C: 1 | V: 0 | IO: 1
--- Banco de Registros ---
R 0: 0x00000000000003E9 | R 1: 0x0000000000000000 | R 2: 0x3FF849DC6894137B | R 3: 0x0000000000000003
R 4: 0x0000000000000009 | R 5: 0x0000000000000000 | R 6: 0x408B980000000000 | R 7: 0x0000000000000000
R 8: 0x3F528E0FA7DD35A3 | R 9: 0x3FF0000000000000 | R10: 0x0000000000000000 | R11: 0x0000000000000000
R12: 0x00000000000003E8 | R13: 0x00000000000000FF | R14: 0x00000000000003E9 | R15: 0x0000000000000000
=====

```

El código del archivo brun.asm tiene como objetivo calcular la suma de los inversos de los números primos gemelos hasta un límite superior de 1.000 que se encuentra almacenado en el registro R12, haciendo uso de las instrucciones definidas se obtiene que se almacena en el R2 el valor hexadecimal que se muestra en la imagen que en decimal corresponde a 1,518 lo cual considerando el relativamente reducido rango de operación se acerca al valor real de la constante (1,902).

```

=====
ESTADO DE LA CPU
=====
PC: 0x2F0 | IR: 0x1200000000000000 | Halted: True
MAR: 0x2E8 | MDR: 0x1200000000000000
Banderas (FR) -> Z: 0 | N: 0 | C: 1 | V: 0 | IO: 1
--- Banco de Registros ---
R 0: 0x00000000000002711 | R 1: 0x0000000000000000 | R 2: 0x3FF9DECBC7648CE3 | R 3: 0x0000000000000003
R 4: 0x0000000000000009 | R 5: 0x0000000000000000 | R 6: 0x40C3658000000000 | R 7: 0x0000000000000000
R 8: 0x3F1A658363C93F70 | R 9: 0x3FF0000000000000 | R10: 0x0000000000000000 | R11: 0x0000000000000000
R12: 0x00000000000002710 | R13: 0x00000000000000FF | R14: 0x00000000000002711 | R15: 0x0000000000000000
=====

```

Consecuente a esto se amplió el rango a 10.000 y se obtuvo el número en decimal aproximado 1,621, lo que evidencia que a mayor rango el resultado almacenado en el R2 se acerca al valor real de la constante, pues abarca una mayor cantidad de números que usa como estudio para buscar si encuentra primos gemelos.

Encontrar la raíz cuadrada de 8 mediante el método de Herón

```
LOAD R8, VALUE_2

LOAD R9, VALUE_8

LDI R10, 12

FHERON R14, R9, R10

FUNPACK R14

MOV R15, R14

HLT
```

VALUE_2:

.quad 0x4000000000000000

VALUE_8:

.quad 0x4020000000000000

```
=====
                        ESTADO DE LA CPU
=====
PC: 0x238 | IR: 0x1200000000000000 | Halted: True
MAR: 0x230 | MDR: 0x1200000000000000
Banderas (FR) -> Z: 0 | N: 0 | C: 0 | V: 0 | IO: 1
--- Banco de Registros ---
R 0: 0x0000000000000000 | R 1: 0x0000000000000000 | R 2: 0x0000000000000400 | R 3: 0x0000000000000001
R 4: 0x0006A09E667F3BCC | R 5: 0x0016A09E667F3BCC | R 6: 0x0000000000000001 | R 7: 0x0000000000000000
R 8: 0x4000000000000000 | R 9: 0x4020000000000000 | R10: 0x000000000000000C | R11: 0x0000000000000000
R12: 0x0000000000000000 | R13: 0x00000000000000FF | R14: 0x4006A09E667F3BCC | R15: 0x4006A09E667F3BCC
=====
```

Usando el método de iteraciones se pretende hallar el resultado de la raíz cuadrada de 8, los registros 14 y 15 almacenan el valor del resultado en formato hexadecimal de doble precisión, R8 almacena el valor de 2, R9 almacena el valor del número 8 y R10 el valor de 12 para establecer el contador de iteraciones. Los registros del 1 al 6 descomponen la estructura del formato estándar para que el emulador los pueda procesar. Finalmente el valor 0x4006A09E667F3BCC corresponde aproximadamente a 2.8284 o $2\sqrt{2}$, lo cual es correspondiente al resultado aproximado esperado de encontrar la raíz cuadrada de 8.

Suma de 2 números en punto flotante

```
LOAD R8, VALUE_1_5
```

```
LOAD R9, VALUE_2_25
```

```
FADD R10, R8, R9
```

```
HLT
```

VALUE_1_5:

```
.quad 0x3FF8000000000000
```

VALUE_2_25:

```
.quad 0x4002000000000000
```

```
=====
                        ESTADO DE LA CPU
=====
PC: 0x220 | IR: 0x1200000000000000 | Halted: True
MAR: 0x218 | MDR: 0x1200000000000000
Banderas (FR) -> Z: 0 | N: 0 | C: 0 | V: 0 | IO: 1
--- Banco de Registros ---
R 0: 0x0000000000000000 | R 1: 0x0000000000000000 | R 2: 0x0000000000000400 | R 3: 0x0000000000000001
R 4: 0x0006A09E667F3BCC | R 5: 0x0016A09E667F3BCC | R 6: 0x0000000000000001 | R 7: 0x0000000000000000
R 8: 0x3FF8000000000000 | R 9: 0x4002000000000000 | R10: 0x400E000000000000 | R11: 0x0000000000000000
R12: 0x0000000000000000 | R13: 0x00000000000000FF | R14: 0x4006A09E667F3BCC | R15: 0x4006A09E667F3BCC
=====
```

Este código simple carga el valor de 1.5 en el registro 8 y 2.25 en el registro 9, ejecuta la instrucción de suma almacenando el resultado en R10 el cual corresponde al número hexadecimal de doble precisión 0x400E000000000000 que en decimal se traduce a 3.75, demostrando que el emulador puede ejecutar operaciones entre números de punto flotante de manera correcta.